Justin Graham

November 11, 2026

IT FDN 110 GP

ASSIGNMENT 05

Introduction

In this document, I discuss the steps taken in completing the fifth course assignment. In this assignment, I learned how to read and write data to a JSON file, collecting data into dictionaries, and basic methods of error handling. I was also tasked with submitting my assignment to a repository on Github. A link to the repository has been posted to the course's discussion board as requested by the assignment criteria. I will also include the URL in this document: https://github.com/GrahamTester/IntroToProg-Python-Mod5

Importing JSON and Setting Constants/Variables

Because the script is working with JSON, the first step in writing the script was in importing the JSON built in to Python. Without doing so, the script will not be able to function as intended.

The constants and variables are very similar to what was asked for in Assignment04. Like the previous assignment, I used the provided starter Python file that already included most of the constants and variables to be used as defined by the acceptance criteria.

For the constants, the only change required to the starter file was renaming "Enrollments.csv" to "Enrollments.json".

```
import json

# Define the Data Constants
MENU: str = '''
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
-----------------------------------------
'''

FILE_NAME: str = "Enrollments.json"
```

The variables used are also very similar to those used in the previous assignment. The one change required by the acceptance criteria is that the student_data variable, which was a list in Assignment04, is now a dictionary. This dictionary, as well as the remaining variables, are defined as empty by default.

```
# Define the Data Variables
student_first_name: str = ''  # Holds the first name of a student entered by the user.
student_last_name: str = ''  # Holds the last name of a student entered by the user.
course_name: str = ''  # Holds the name of a course entered by the user.
student_data: dict = {}  # one row of student data
students: list = []  # a table of student data
file = None  # Holds a reference to an opened file.
menu_choice: str = ''  # Hold the choice made by the user.
```

Reading Data on Program Start

One of the most significant changes in writing the code for Assignment05 compared to the previous assignment was in writing the code to read data from an existing JSON file on start. With the JSON library imported, it is possible to read data from the file with a line such as 'students = json.load(file)', which loads the contents of the defined json file into memory.

For this assignment, it was also requested that error handling be conducted on this piece of code. WIth the syntax of 'try' and 'except', the script tries to execute the code, and if an error occurs, it follows the except logic. It is also possible to set more than one 'except' line to account for different potential error states.

For this section of code, I included a FileNotFoundError that triggers if the required JSON file is missing. There is also a more general exception to catch other errors if something else is wrong.

This error check ends with the 'finally' logic, where if the code sees that the JSON file is present, but not closed, it will close the file.

```
# When the program starts, read the file data into a list of dictionaries (table)
# Extract the data from the file
try:
    file = open(FILE_NAME, "r")
    students = json.load(file)
    file.close()
except FileNotFoundError as e:
    print("File not found.")
except Exception as e:
    print("An error occurred while attempting to open the JSON file.")
    print(e, e.__doc__)
finally:
# If error occurred, check if a file object exists and is still open
    if file is not None and file.closed == False:
        file.close()
```

Loop Initialization

The next piece of code is unchanged from Assignment04. It initiates the while loop and displays the MENU string, presenting the user with four numbered options. Also unchanged from prior assignments, these options, in order, allow the user to input data, display previously input data, save data to the JSON file, and close the program.

After the menu is displayed, the user is asked to input a menu_choice matching one of the items outlined in the MENU string. The loop is initialized with the line while True'.

```
# Present and Process the data
while True:

    # Present the menu of choices
    print(MENU)
    menu_choice = input("What would you like to do: ")
```

Data Input

As in the previous assignment, the program is designed to expect a menu_choice input of 1, 2, 3, or 4. An if-else conditional is used within the while loop to define what to do for each input.

If '1' is input, the following code will run:

```
    # Input user data
    if menu_choice == "1":  # This will not work if it is an integer!
        try:
            student_first_name = input("Enter the student's first name: ")
            if not student_first_name.isalpha():
                raise ValueError("First name can only contain letters.")
            student_last_name = input("Enter the student's last name: ")
            if not student_last_name.isalpha():
                raise ValueError("Last name can only contain letters.")
        except ValueError as e:
            print("Student names can only contain letters.")
            continue
        course_name = input("Please enter the name of the course: ")
        student_data = {"FirstName": student_first_name,
                        "LastName": student_last_name,
                        "CourseName": course_name}
        students.append(student_data)
        print(f"You have registered {student_first_name} {student_last_name}"
              f" for {course_name}.")
        continue
```

As in the previous assignment, the program asks the user to input a student_first_name, student_last_name, and course_name. Unlike in the prior assignment, student_data is a dictionary, and thus concatenates the input as a dictionary with labels for each data point, defined as “FirstName”, “LastName”, and “CourseName”. Once student_data has been defined, it is appended to the ‘students’ list as a dictionary row.

The acceptance criteria asked that error handling be included for the input of student_first_name and student_last_name. Using the try-except logic previously mentioned above, the code checks if there are non-alphabetic characters in the strings input for these fields. If either student_first_name or student_last_name has non-alphabetic characters, a ValueError is thrown, and the user is informed that names can only contain letters.

The code is arranged such that if either the user has input valid data, or the program detects invalid data entry, a continue function loops the code back to the start of the while loop, and the user is asked to input another menu_choice.

Data Display

If the user inputs a menu_choice of 2, the if-else conditional advances to the second defined option. When run, the code enters a for loop for each student in the ‘students’ list and prints the data for the user to read in a formatted string that uses the student_data dictionary labels to define how the data is arranged.

The program then loops back to the start, asking the user to input a menu_choice.

```
# Present the current data
elif menu_choice == "2":

    # Process the data to create and display a custom message
    print("-" * 50)
    try:
        for student in students:
            print(f"Student {student['FirstName']} "
                  f"{student['LastName']}"
                  f" is enrolled in "
                  f"{student['CourseName']}.\n")
    except Exception as e:
        print("An error occurred while attempting to display"
              " student data.")
        print(e, e.__doc__)
        continue

    print("-" * 50)
    continue
```

As seen in this screenshot, there is also error handling code present here. I inserted it while debugging the assignment, as I encountered issues with displaying the requested string. The issues stemmed from mistakes made earlier in the script, where the data had not been read into the file properly. In my debugging of the error, I had first attempted to format the code for menu_choice 2 in several different ways, including the correct manner above, but due to the true problem, the code did not work until I identified the source of the issue with the aid of reviewing a classmate's work on Github.

I have left the error handling code in place to illustrate how I arranged it while the error still occurred.

Saving the Data

Selecting menu_choice 3 writes the contents of the 'students' list to the Enrollments.json file.

Similar to the code run when the program first starts, the Enrollments.json file is opened, but in ‘write’ mode. However, unlike with writing to a csv, the function json.dump() is used instead. Inside the parentheses are the list ‘students’, the file’ variable, and an instruction to indent the data for cleaner JSON formatting.

Once the data is written to the JSON file, the program informs the user that the data is saved to the file. The contents saved include all data that existed in the json file when the program was opened and any data added via menu_choice 1.

The acceptance criteria also requested error handling for this stage as well. On writing the data to the JSON file, the program first checks for a TypeError to ensure that the file is the appropriate format. It then checks for more general exceptions. If an error is found, and a file exists, the file is then closed.

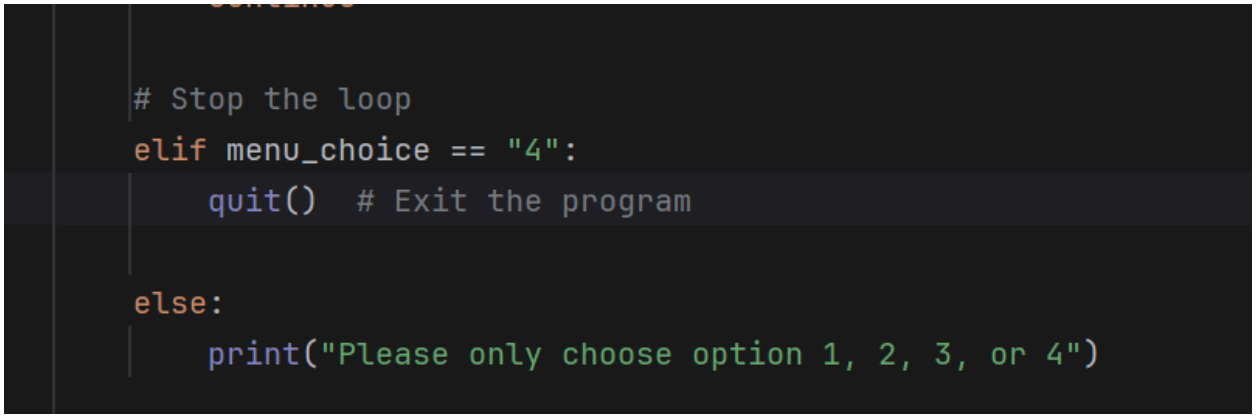
As included in the screenshot below, there is additional error handling around the display of the text listing out the data saved to the file. The same error that affected menu_choice 2 above also happened in this section of the menu_choice 3 code, meaning that it would successfully save the data but then fail on printing the message listing the data saved. As such, I included error handling while investigating the bug and have left the code in the assignment to illustrate the step taken.

```
# Save the data to a file
elif menu_choice == "3":
    try:
        file = open(FILE_NAME, "w")
        json.dump(students,file,indent=2)
        file.close()
    except TypeError as e:
        print("Please ensure data is in JSON format.\n")
        print(e, e.__doc__)
    except Exception as e:
        print("An error occurred while writing to the file.\n")
        print(e, e.__doc__)
    finally:
        # If error occurred, check if a file object exists and is still open
        if file is not None and file.closed == False:
            file.close()

    try:
        print("The following data was saved to file!")
        for student in students:
            print(f"Student {student['FirstName']} "
                  f"{student['LastName']} "
                  f"is enrolled in {student['CourseName']}.\n")
    except Exception as e:
        print("Data is saved. An error occurred displaying string.\n")
        print(e, e.__doc__)
        continue
    continue
```

Exiting the Program and Handling Bad Input

As in the previous assignment, if ‘4’ is input as the ‘menu_choice’, the program ends. If the user input an invalid menu_choice (ie. anything other than 1, 2, 3, or 4), it reminds the user to input a valid menu choice.



```
# Stop the loop
elif menu_choice == "4":
    quit()  # Exit the program

else:
    print("Please only choose option 1, 2, 3, or 4")
```

JSON Data

After running the program, inputting data, and saving the list contents, the output in the resultant JSON file resembles the following:



Summary

Throughout this document, I have mentioned the error-handling code written, both as asked for per the acceptance criteria, as well as code I wrote to clean up error behavior for bugs encountered in the act of debugging. This was an excellent lesson in the basics of including error handling and as I get more accustomed to it, I'm sure it will be of great help. I also have to commend my classmates and their code for helping me realize where I had made my mistakes. Perhaps the most challenging part of the assignment was in just getting used to the relative simplicity of JSON compared to reading from and writing to csv files. The core of the issues

encountered stemmed from mistakenly leaving in code for reading csv data, and it took me too long to realize where the error truly was because I was too focused on the wrong part of the code. It's a good lesson to be more mindful of the code as a whole.